

FIELD PROJECT

FitHub — Gym Management Website Development

Department of Information Technology

MEMBERS :

SHRADDHA MAHINDRAKAR 177

PRIYANKA MANE 179

SUMEET MORE 184

KHUSHI RAUT 191

Technology: MERN Stack — MongoDB · Express.js · React · Node.js

Live: <https://fithub-api-3wjz.onrender.com>

CHAPTER 1 : INTRODUCTION

1.1 Background

Most local gyms are still run using paper notebooks and messy registers, which makes tracking memberships and daily operations a major headache and can easily lose the track of the gym. We built FitHub to fix these problems by replacing chaotic paperwork with a single, simple web platform. The system gives administrators complete control, allowing them to oversee the entire facility, track member statuses, and handle payments all from one secure dashboard. By bringing everything into one digital space, FitHub keeps the business organized.

1.2 Problem statements

Running a gym with paper notebooks and registers is slow, messy, and leads to easy mistakes. Owners constantly lose track of whose membership is expiring, who still owes money, etc. Because all the information is scattered, it is almost impossible to quickly check monthly income or see if the business is actually growing. This manual paperwork creates daily chaos and makes it really hard to run the gym smoothly as more members join.

1.3 Purpose of project

- Get rid of paper registers: The gym currently uses only paper notebooks and diaries, which are messy, take up space, and can easily get lost or ruined.
- Fix manual mistakes: Right now, the staff has to flip through pages by hand just to see who paid, who is new, or whose membership is ending, which takes way too long.
- Give the gym its very first online system: This gym has absolutely no digital platform right now, so this project moves them from old-school paperwork to the internet for the first time.
- Put everything in one place: Instead of checking different notebooks, the owner can now see members and fees all on one simple computer screen.
- Save time and effort: By stopping the boring paperwork, the gym staff can stop stressing over daily records and focus on running a better fitness center.

1.4 Objective of project

- To build a simple web application called FitHub that replaces all physical registers and paper notebooks with a digital system.
- To create a central dashboard where the gym admin can view and manage members, and payments all in one place.
- To automate the tracking of membership expiration dates and pending fees so the staff doesn't have to check records by hand.
- To secure the gym's data by setting up proper login accounts for administrators and members.

- To provide the gym with its very first online platform, moving their entire business from manual files to the internet.

1.5 Scope of the Project

The scope of this project is to build a web-based platform called FitHub specifically for a local gym that currently uses manual paper registers. The system moves all member records, membership packages, and transactions online. It provides secure login access for both the administrator and gym members, allowing the gym to process digital payments and communicate with members automatically, making daily operations entirely paperless.

1.5.1 Features:

Secure login system for admins and gym members.

Digital registration forms to quickly sign up new members.

Dynamic package creation for different gym membership tiers.

Integrated secure online checkout for membership fees.

Instant email receipts sent automatically upon payment confirmation.

Automated text and email alerts for upcoming membership expiration dates.

Search bar to instantly find any member's details by name or phone number.

In-gym product store with cart, orders and order receipts.

Member progress tracking (weight and body measurements) with charts.

Member reviews and ratings with admin moderation.

Gym calendar for classes, events and holidays, plus homepage banners.

1.5.2 Exclusions

Trainer Management: Registering gym trainers and assigning them to member schedules or slots.

Workout & Diet Planners: Custom daily exercise and meal charts for members.

Mobile Application: Building dedicated Android or iOS apps for easier mobile access.

Biometric/ID Attendance: Linking fingerprint scanners or QR codes at the gym door to the software.

CHAPTER 2 : LITERATURE SURVEY

System Name : Cult for Corporates (by Cult.fit).

Brief Description :

It is an all-in-one, B2B digital and physical health platform designed to transform traditional company wellness into a high-engagement fitness culture. Instead of just offering basic gym memberships, it provides companies with customized fitness programs, flexible gym network access, and analytics tools to track employee health, boost morale, and increase productivity.

Feature List :

- Premium Fitness Access (Cultpass): Employees get flexible access to over 2,000 physical gym facilities, Cult centers, and online home-workout formats.
- Diverse Workout Formats: Access to structured group classes including Yoga, Dance Fitness, Strength & Conditioning (S&C), HRX workouts, Boxing, and Pilates.
- Exclusive HR Dashboard: A centralized tracking tool for corporate HR departments to monitor overall employee participation, engagement levels, and health trends.
- On-Campus / Onsite Solutions: Expert-led live health sessions, dance, or yoga events organized directly inside the company's office space.
- Corporate Gamification & Challenges: Inter-company or intra-company sports leagues (Cricket, Football, Badminton) and weight loss/fitness challenges to build team spirit. Dietician & Expert Consultations: One-on-one virtual or physical guidance from certified nutritionists, physiotherapists, and mental health coaches.
- Corporate Gifting & Rewards: Customized reward programs and health vouchers to recognize and appreciate active employees ("Workplace Heroes").

Technology Overview:

- Hardware Mobile & Desktop Devices: Smartphones (iOS and Android) and computers used by employees and HR admins to access the platform.
- Smart Gym Infrastructure: Connected QR code scanners and biometric check-in systems at physical Cult/partner centers to log attendance.
- Wearable Integration: Capability to sync data from consumer fitness hardware (such as smartwatches, fitness bands, and phone step-counters) to the mobile app.
- Software Cult.fit Mobile App: The user-facing software where employees book class slots, join virtual live streams, track personal health goals, and pause subscriptions.
- HR Analytics Web Portal: A secure SaaS (Software-as-a-Service) dashboard built for company administrators to download reports and view employee engagement ROI.
- Cloud Infrastructure: Back-end servers and databases (often hosted on AWS or similar cloud systems) that handle massive real-time booking traffic and securely store user details.

- Privacy & Compliance Protocols: Strict enterprise data-security frameworks built to satisfy data protection compliance standards (e.g., ISO 9001 and ISO 45001 certification).
- Slot Booking & Inventory Optimization Algorithms: Real-time scheduling systems that fairly allocate gym and class capacities based on demand, cancellations, and waiting lists.
- Personalized Recommendation Engine: Machine learning algorithms within the app that suggest workout routines, intensity levels, and diet plans based on an employee's goals and past activity.
- Engagement & Analytical Aggregation: Algorithms that anonymize and group employee health data to generate broad analytics for HR dashboards while strictly protecting individual privacy.

System Name : GYM IT Fitness System.

Brief Description :

The GYM IT Fitness System is a data-driven fitness management platform. It is built to automate gym operations across multiple branches. The platform connects gym administration, personal trainers, and members into one unified ecosystem to track health progress.

Feature :

- Multi-Branch Dashboard: Manages distinct gym locations from a single software interface.
- Data-Driven Tracking: Records fitness metrics to map progress and establish future goals.
- Integrated Lead Management: Captures customer inquiries through web contact forms.
- Personalized Programs: Distributes specialized schedules for activities like Weight Loss, MMA, and Yoga.
- Nutritional Guidance Portal: Integrates customized diet plans to complement daily workout routines.
- Built-in BMI Calculator: Allows quick body mass index evaluations directly on the website.
- Review Aggregator: Displays real-time Google Reviews to establish credibility and gather user feedback.

Technological Overview:

- Hardware Client Devices: Smartphones, tablets, and desktop computers used by members and staff.
- Cloud Infrastructure: Remote physical servers that host the gym database and website files safely.
- Software Front-End (User Interface): Built using standard HTML, CSS, JavaScript, and Bootstrap frameworks to ensure the layout functions smoothly on both computers and phones.
- Back-End (Logic): Programmed with languages like PHP or Node.js to securely process member sign-ups, form submissions, and calculator operations.
- Database Management: Uses MySQL or cloud databases to systematically store member profiles, trainer allocations, and operational logs.
- Mathematical Calculators: Employs algebraic logic to process user weight and height measurements into a precise Body Mass Index score ($BMI = \text{weight} / \text{height}^2$).
- Data Analytics: Filters and tallies structural progress metrics over time, giving clear visualization charts to spot health trends.
- Review Sorting Logic: Employs rating algorithms to compute and display the aggregate score of the business.

2.3 Comparative Analysis – Feature Gap Table

The table below compares the two existing systems studied above with the proposed FitHub system, highlighting the feature gaps in each and how FitHub is designed to fill them.

Feature Gap Identified	Cult for Corporates	GYM IT Fitness System	How FitHub Bridges the Gap
Online purchase & renewal of individual membership plans	Not available – platform is B2B/corporate-only; no direct individual purchase flow	Not available – uses a contact/lead form instead of an online purchase flow	Provides a complete View → Compare → Buy/Renew Membership flow for individual users
Integrated online payment with receipt generation	No dedicated payment/receipt module described	Not available	Integrates online payment with instant confirmation and a downloadable receipt
Self-service account recovery (forgot/reset password)	Not documented	Not documented	Includes a secure, email-based forgot/reset password flow
Dedicated admin panel for payment approval & member management	HR dashboard exists but is analytics-only, not payment/membership focused	No dedicated admin approval workflow	Gives admins a dashboard to manage members, approve payments, and send notifications
Membership & payment history tracking for the user	Not focused on individual membership history	Not available	Lets users view their full membership and payment history anytime
Affordable, single-gym scale solution	Designed for large corporate clients; not cost-effective for a single gym	Enterprise, multi-branch focus; complex/overkill for one gym	Lightweight and purpose-built for a single gym's day-to-day operations

CHAPTER 3 : SYSTEM REQUIREMENTS AND DESIGN

3.1 System Modules

The FitHub system is divided into four major modules, each handling a distinct part of the gym membership management process: the User Module, Membership Module, Payment Module, and Admin Module — plus supporting modules for the store, progress tracking, reviews, calendar and banners. Each module and its step-by-step working is explained below.

3.1.1 User Module

This module handles everything related to a user's identity and account on the platform.

Registration:

- The new user opens the FitHub registration page and enters details such as name, email address, phone number and password (with confirm password).
- The frontend validates the input fields (e.g., valid email format, 10-digit Indian mobile number, minimum 8-character password, matching passwords, required fields).

- On submission, the data is sent to the backend registration API (`POST /api/auth/register`).
- The server checks whether the email is already registered. If it exists, an error is returned.
- If the email is new, the password is hashed with bcrypt and the user record is stored in MongoDB.
- An email OTP is issued for account verification, and the user is redirected to the Login page with a success message.

Login:

- The registered user enters their email and password on the login page.
- The frontend checks that both fields are filled (empty fields trigger a "required" message).
- Valid input is sent to the backend login API (`POST /api/auth/login`) as a JSON request.
- The server verifies the credentials against the stored (hashed) password with bcrypt.
- If valid, a signed JWT token is created and the user is redirected to the dashboard (members) or admin panel (role = admin); if invalid, an error message is displayed.

Forget Password:

- The user clicks "Forgot Password" on the login page.
- The user enters their registered email address.
- The system generates a time-limited email OTP and sends it to the user's email.
- The user enters the OTP and sets a new password.
- The new password is validated and updated in the database (re-hashed with bcrypt).

Profile Management:

- The logged-in user navigates to the "My Profile" section.
- Existing details (name, phone, date of birth, gender, address, emergency contact) are fetched from the database and displayed.
- The user edits the required fields and clicks "Save"/"Update".
- The updated data is validated and saved back to the database.

3.1.2 Membership Module

This module allows users to browse, purchase, and track gym membership plans.

View Membership Plans:

- The user opens the "Membership Plans" page.
- The system fetches all active plans (name, duration + unit, price, features, popular flag) from the database (`GET /api/subscriptions/all`) and displays them as cards.

Compare Plans:

- The user selects two or more plans to compare.
- The system displays a side-by-side table of features, duration, and price for easy comparison.

Buy Membership:

- The user selects a plan and clicks "Buy Membership".
- The system checks whether the user is logged in; if not, the user is redirected to login first.
- The user confirms the plan and chooses online payment (Razorpay Checkout) or pay-at-gym.
- Once payment succeeds, a new membership record is created with start date, end date (auto-calculated from duration), and "active" status (`POST /api/members/:id/membership`).

Renew Membership:

- An existing member opens "Renew Membership" from their dashboard, usually shown before/after expiry.
- The user selects the same or a new plan and proceeds to payment.
- On successful payment, the membership end date is extended and status is updated.

Membership History:

- The user opens "Membership History" / "Transactions" to view all past and current memberships along with payment status, amounts paid/remaining and receipts.

3.1.3 Payment Module

This module manages the financial transactions of the platform.

Online Payment:

- After selecting a plan, the user is directed to the payment page.
- The user pays through Razorpay Checkout (card/UPI/net-banking) or chooses pay-at-gym.
- The payment request is sent securely to the gateway/backend for processing and the Razorpay signature is verified on the server (`POST /api/payment/verify`).

Payment Confirmation:

- The payment gateway/backend validates the transaction.

- On success, the payment status is marked "paid" and linked to the corresponding membership.
- On failure, the status is marked "failed" and the user is prompted to retry.
- Partial / installment payments are supported: the admin records each installment and the server rejects any amount exceeding the remaining balance (HTTP 400 overpay guard).

Download Receipt:

- Once payment is confirmed, the system generates a receipt containing transaction ID, amount, date, method and plan details.
- The user can view, print or download this receipt from their dashboard/history, and a copy is emailed automatically on every purchase or recorded installment.

3.1.4 Admin Module

This module gives the gym administrator control over users, plans, and payments.

Manage Members:

- The admin logs into the Admin Dashboard and opens the "Members" section.
- The admin can view all members with search, add new members (optionally enrolling them in a plan in the same step), edit details, change/assign plans, or remove member accounts.

Manage Membership Plans:

- The admin opens "Manage Plans" to add a new plan (name, duration, price, features, popular flag) or edit/deactivate an existing plan.

Approve Payment:

- The admin reviews paid/partial/pending memberships in the "Payments" section (`GET /api/members` with payment totals).
- The admin records cash/UPI/card installments per member (`POST /api/members/:id/payment`) and the ledger updates instantly.

Send Notification:

- The system automatically emails a branded payment receipt on every purchase or installment.
- A scheduled job (every 6 hours) emails members upcoming-expiry reminders and flags them so nobody is reminded twice.

View Reports:

- The admin opens the dashboard to view total members, revenue collected, active/expiring/expired membership counts, and the full transaction ledger with per-record receipts.

3.1.5 Supporting Modules

- **Store & Orders** — product catalog with stock tracking, cart, checkout (Razorpay/cash), order status and order receipts.
- **Progress Tracking** — members log weight and body measurements over time with charts.
- **Reviews** — members submit ratings/comments; admin approves or hides them.
- **Calendar & Banners** — admin-published gym classes, events, holidays and homepage banners.

Key REST Endpoints Used

Method & Endpoint	Purpose
<code>POST /api/auth/register</code>	Register a new member (validated, bcrypt-hashed)

POST /api/auth/login	Login, returns signed JWT (rate-limited 15 / 15 min)
POST /api/auth/send-otp · POST /api/auth/reset-password	Email OTP issue and password reset
GET /api/subscriptions/all	List active membership plans
GET /api/members (admin)	List members with membership + validity info
POST /api/members/:id/membership (admin)	Assign / renew a membership plan
PUT /api/members/:id · DELETE /api/members/:id (admin)	Edit / remove a member
POST /api/members/:id/payment (admin)	Record an installment (overpay rejected: 400)
GET /api/members/all-transactions (admin)	Full payment ledger with totals
POST /api/payment/verify	Verify Razorpay signature, activate membership
GET /api/orders · POST /api/orders	Store orders with payment tracking

3.2 Hardware and Software Requirements

The requirements below reflect the technology stack actually used to build and host FitHub: a React single-page frontend calling an Express REST API, with MongoDB Atlas as the cloud database. The complete system is deployed as one service on Render, which serves both the API and the production frontend build (live at <https://fithub-api-3wjz.onrender.com>).

3.2.1 Software Requirements

Frontend

Component	Technology	Purpose
UI Library	React 18 (Create React App)	Component-based pages: home, plans, checkout, dashboard, admin panel
Routing	React Router v6	Client-side navigation with protected member/admin routes
Styling	Tailwind CSS 3 + custom CSS	Responsive layout, gym theme, print stylesheet for receipts
HTTP Client	Axios (JWT interceptor)	REST API calls with automatic auth header + 401 redirect
Animations	Framer Motion	Page/modal transitions and micro-interactions
Charts	Recharts	Progress graphs and admin visualizations
Notifications / Icons	react-hot-toast, react-icons, MUI icons	Toast feedback and iconography
Checkout SDK	Razorpay Checkout (checkout.js)	Secure in-browser online payment popup
Browser	Google Chrome / Microsoft Edge (latest)	Rendering and testing the application
Code Editor	Visual Studio Code	Writing and debugging code
Package Manager	npm	Dependency management (incl. qs security override)
Version Control	Git + GitHub	Source control and collaboration
Dev Proxy	CRA proxy → http://localhost:5000	Frontend-to-API calls during local development

Backend

Component	Technology	Purpose
Runtime	Node.js (≥ 16)	Executing server-side JavaScript
Framework	Express.js 4	Building REST APIs (e.g., /api/auth/login)
Database	MongoDB Atlas (NoSQL cloud)	Storing users, memberships, payments, orders, products

ODM	Mongoose 8	Schemas, validation and population of references
Authentication	bcryptjs (hashing), jsonwebtoken (JWT)	Secure password storage and stateless session handling
OTP Verification	Email OTP with expiry (otps collection)	Account verification and password reset
Online Payments	Razorpay SDK (orders + signature verification)	Secure membership and store checkout
Email Delivery	Gmail OAuth2 → Brevo → SendGrid → Resend (failover chain)	Receipts, OTPs and expiry reminders
SMS Provider	Twilio SDK	Text-message notification channel
Security	Helmet (CSP), express-rate-limit, CORS allowlist, express-mongo-sanitize, hpp, 2 MB body limit	Hardening against XSS, injection, abuse and DoS
Scheduler	Recurring background job (6-hour interval)	Membership-expiry email reminders
Hosting	Render (single web service)	Hosts API + serves production frontend build
API Testing	Postman + live endpoint verification	Testing backend endpoints during development

3.2.2 Hardware Requirements

Minimum Hardware Configuration

Component	Client Machine	Server (Cloud)
Processor	Dual-core 1.6 GHz or higher	Managed by Render (shared CPU on starter tier)
RAM	4 GB	512 MB – 2 GB instance + MongoDB Atlas M0 shared cluster
Storage	20 GB free space	Ephemeral instance disk; persistent data lives in Atlas
Internet	Stable broadband connection	Stable broadband connection
Display	1366x768 resolution or higher	Not required (headless server)

3.3 Planning and Scheduling

FitHub was developed over a period of approximately one month (30 days). The project was divided into seven phases. The Gantt chart below shows the planned schedule against the actual schedule followed during development.

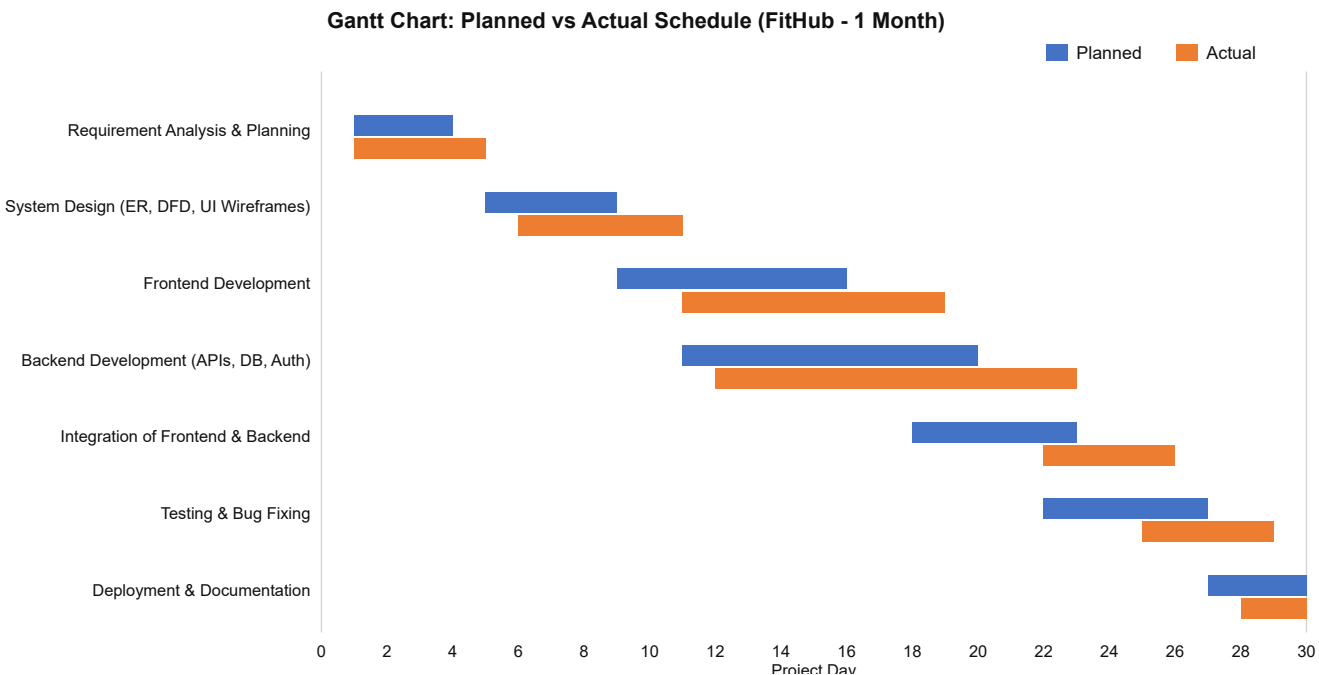


Fig 3.3: Gantt Chart – Planned vs Actual Schedule

As seen in the chart, Backend Development and Testing & Bug Fixing overran their planned duration slightly, which is common when integration issues (such as CORS/origin errors between the frontend and the hosted API) surface only during the integration phase.

3.4 Conceptual Module

The conceptual design of FitHub is represented using an Entity-Relationship (ER) diagram, showing the core entities and their relationships.

FitHub — Entity-Relationship Diagram

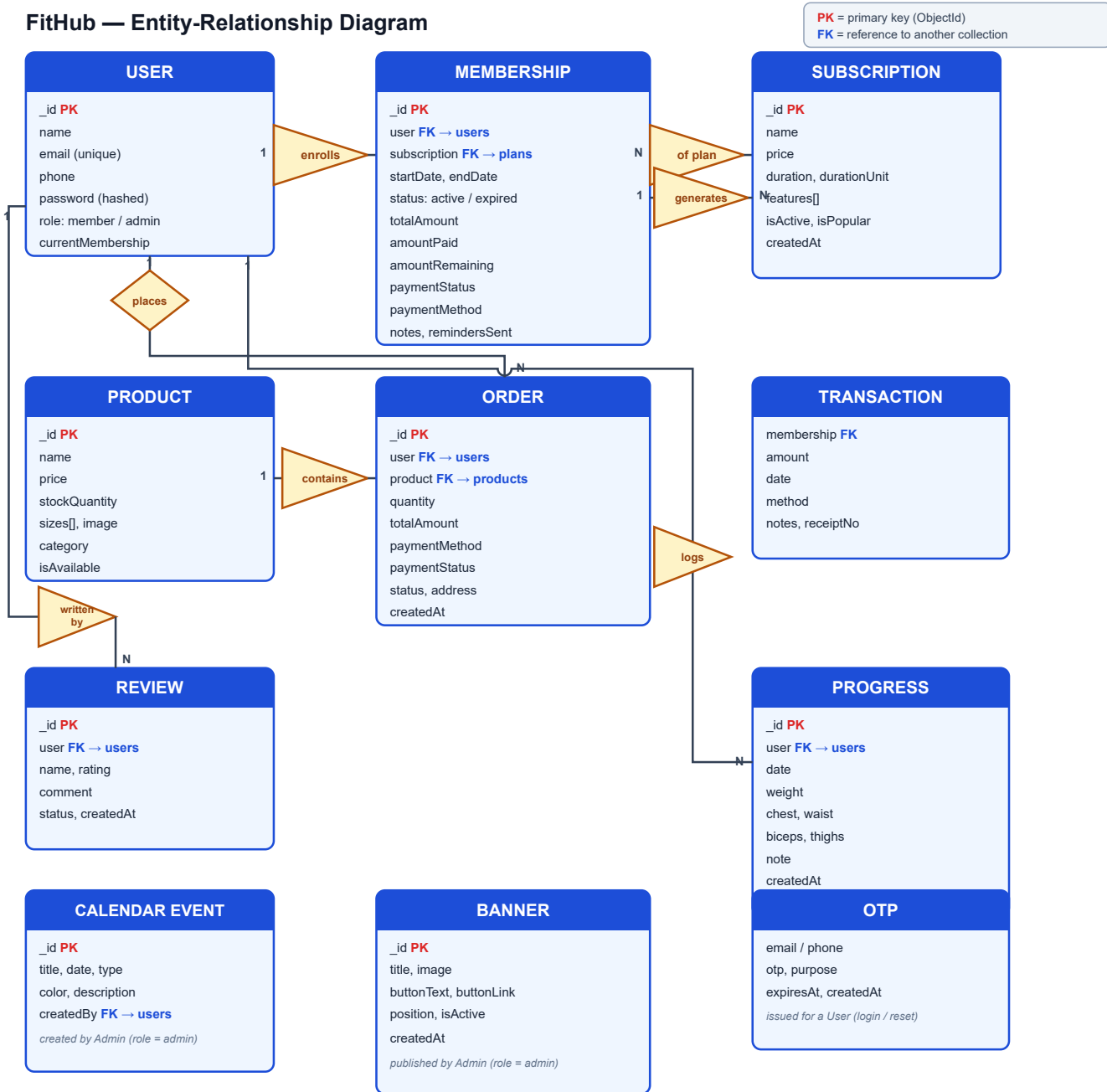


Fig 3.4: Entity-Relationship (ER) Diagram of FitHub

3.4.1 Data Dictionary

The data dictionary below lists the metadata (fields, data types, and descriptions) for each entity shown in the ER diagram above. FitHub uses MongoDB, so keys are ObjectId values generated by the database and references between collections.

Table: User (collection: users — members and admins, told apart by role)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each user
name	String	-	Full name of the user
email	String	Unique	Login identifier / contact email
password	String (bcrypt hash)	-	Hashed password, never stored in plain text

phone	String	-	10-digit contact number
role	String	-	member or admin (admin owns the panel)
isVerified	Boolean	-	Email OTP verification flag
profileImage	String	-	Path/URL to profile picture
address	String	-	Residential address
dateOfBirth	Date	-	Date of birth
gender	String	-	Gender
emergencyContact	String	-	Emergency contact number
currentMembership	Embedded object	-	Snapshot of the active membership for fast reads
createdAt	Date	-	Account creation timestamp

Table: Subscription (collection: subscriptions — membership plans)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each plan
name	String	-	Name of the plan (e.g., Basic, Premium)
description	String	-	Plan summary shown on the plans page
duration	Number	-	Validity length of the plan
durationUnit	String	-	Unit of duration (days / months)
price	Number	-	Price of the plan in rupees
features	Array of Strings	-	Features included in the plan
isActive	Boolean	-	Whether the plan is on sale
isPopular	Boolean	-	Highlights the plan as "Most Popular"
createdAt	Date	-	Plan creation timestamp

Table: Membership (collection: memberships)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each membership record
user	ObjectId	FK → users._id	Member who owns the membership
subscription	ObjectId	FK → subscriptions._id	Plan purchased
startDate	Date	-	Membership start date
endDate	Date	-	Membership expiry date (auto-calculated)
status	String	-	active / expired / renewed
totalAmount	Number	-	Full price of the plan
amountPaid	Number	-	Total paid so far (supports installments)
amountRemaining	Number	-	Balance still due
paymentStatus	String	-	paid / partial / pending
paymentMethod	String	-	razorpay / cash / upi / card
notes	String	-	Admin notes (e.g., renewal remarks)
emailRemindersSent	Number	-	Count of expiry reminders already emailed
createdAt	Date	-	Record creation timestamp

Table: Transaction (payment ledger entry per membership)

Field Name	Data Type	Key	Description
membership	ObjectId	FK → memberships._id	Membership this payment belongs to
amount	Number	-	Amount paid in this transaction

date	Date	-	Date and time of payment
method	String	-	razorpay / cash / UPI / card
notes	String	-	Remarks recorded with the payment
receiptNo	String	-	Unique receipt number on the receipt

Table: Product (collection: products — gym store)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each product
name	String	-	Product name
description	String	-	Product details
price	Number	-	Selling price in rupees
stockQuantity	Number	-	Units available in stock
category	String	-	Product category
image	String	-	Path/URL of product image
sizes	Array of Strings	-	Available sizes (e.g., S, M, L, XL)
isAvailable	Boolean	-	Whether the product can be ordered
createdAt	Date	-	Product creation timestamp

Table: Order (collection: orders)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each order
user	ObjectId	FK → users._id	Member who placed the order
product	ObjectId	FK → products._id	Product ordered
quantity	Number	-	Units ordered
totalAmount	Number	-	Order total in rupees
paymentMethod	String	-	razorpay / cash / UPI / card
paymentStatus	String	-	paid / pending / failed
status	String	-	Order fulfilment status
shippingAddress	String	-	Delivery address
createdAt	Date	-	Order placement timestamp

Table: Review (collection: reviews)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each review
user	ObjectId	FK → users._id	Member who wrote the review
name	String	-	Display name on the review
rating	Number (1–5)	-	Star rating
comment	String	-	Review text
status	String	-	approved / hidden (admin moderation)

createdAt	Date	-	Review submission timestamp
-----------	------	---	-----------------------------

Table: Progress (collection: progress — body measurements)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each log entry
user	ObjectId	FK → users._id	Member logging progress
date	Date	-	Measurement date
weight	Number	-	Body weight in kg
chest / waist / biceps / thighs	Number	-	Body measurements in inches
note	String	-	Optional remarks
createdAt	Date	-	Log creation timestamp

Table: CalendarEvent (collection: calendarevents)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each event
title	String	-	Event / class name
description	String	-	Event details
date	Date	-	Event date
type	String	-	event / class / holiday / other
color	String	-	Calendar display color
createdBy	ObjectId	FK → users._id	Admin who published the event

Table: Banner (collection: banners)

Field Name	Data Type	Key	Description
_id	ObjectId	PK	Unique identifier for each banner
title	String	-	Banner headline
description	String	-	Banner sub-text
image	String	-	Path/URL of banner image
buttonText / buttonLink	String	-	Call-to-action label and target
position	String	-	Placement slot on the site
isActive	Boolean	-	Whether the banner is shown
createdAt	Date	-	Banner creation timestamp

Table: OTP (collection: otps — one-time passwords)

Field Name	Data Type	Key	Description
email / phone	String	-	Account the OTP was issued for
otp	String	-	The one-time code
purpose	String	-	verify-account / reset-password
expiresAt	Date	-	Expiry time of the code
createdAt	Date	-	Issuance timestamp

Note: the administrator is not a separate table — admin access is a `role = "admin"` value on the User collection. Email notifications (receipts, OTPs, expiry reminders) are generated from Membership/Transaction records by the backend and delivered through the email failover chain, so no separate notification table is stored.

3.4.3 DFD (Data Flow Diagram)

Level 0 (Context Diagram): shows FitHub as a single process interacting with the User and Admin.

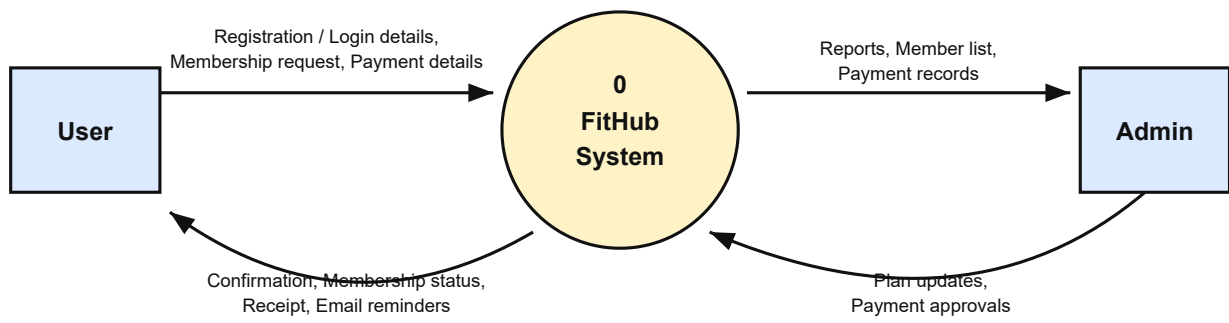


Fig 3.4.3(a): DFD Level 0 – Context Diagram

Level 1: breaks the system down into its four core processes and the data stores each one reads from or writes to.

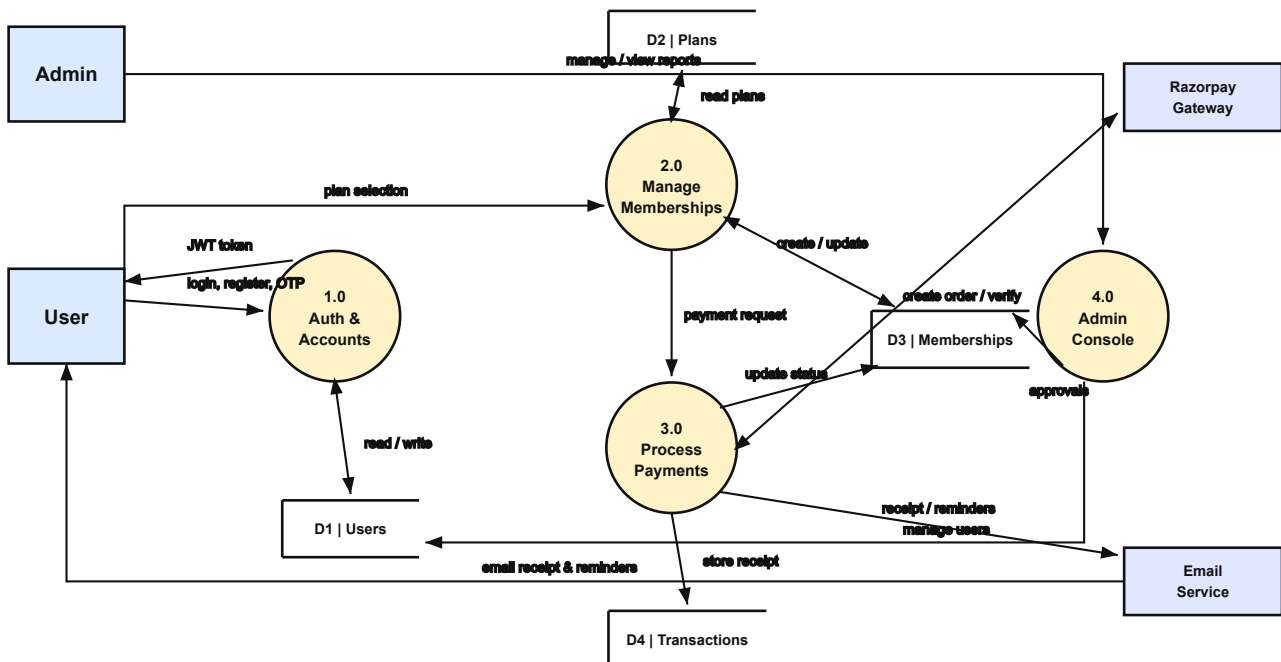


Fig 3.4.3(b): DFD Level 1

Admin Console (4.0) reads and writes all data stores D1–D4; cash/UPI payments collected at the gym are approved and recorded here.

3.4.4 Use Case Diagram

The use case diagram below identifies two actors — User and Admin — and an external Payment Gateway, along with the actions each actor can perform on the system.

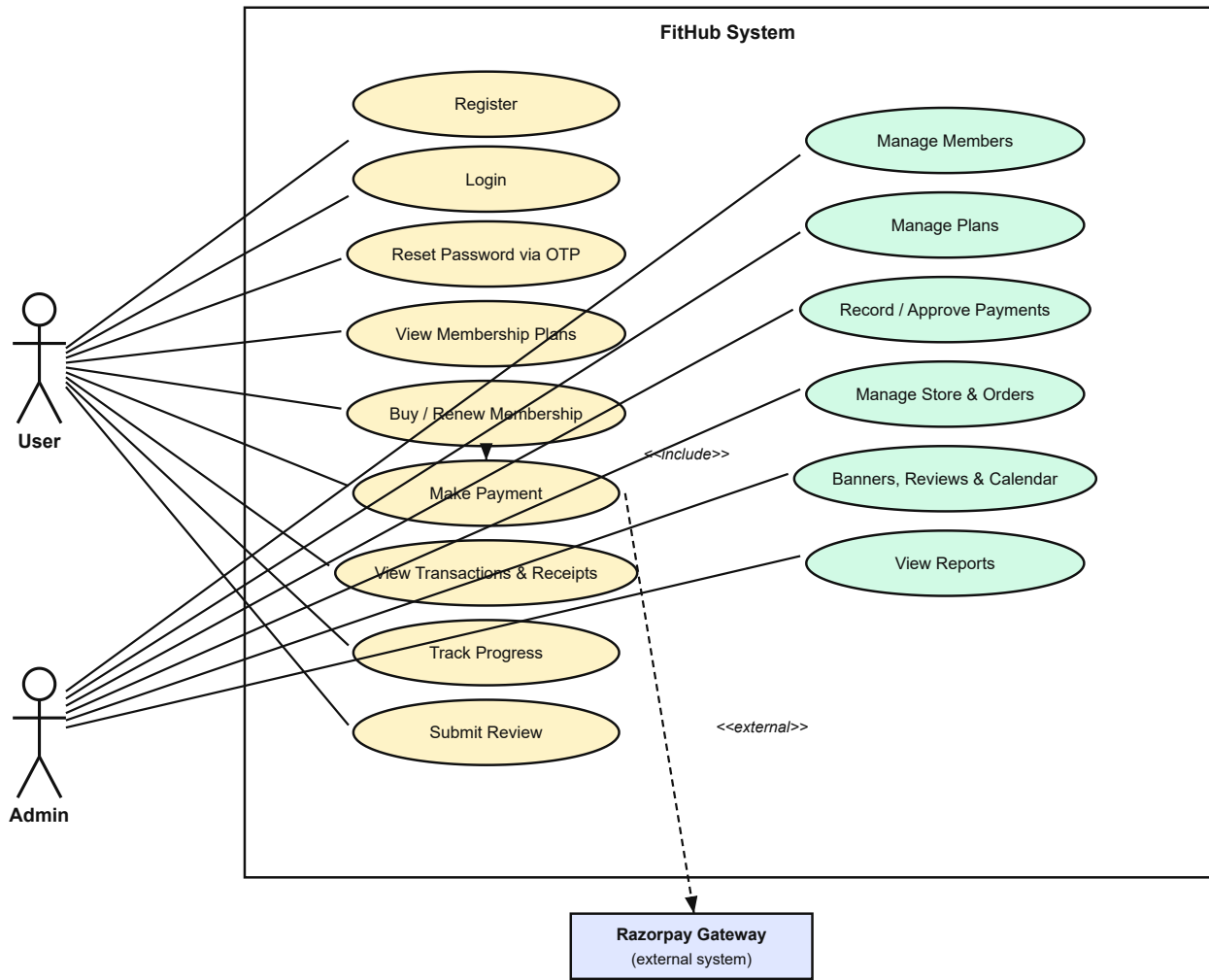


Fig 3.4.4: Use Case Diagram of FitHub

3.4.5 Activity Diagram

The activity diagram below illustrates the flow of the "Buy Membership" process, from plan selection through payment to receipt generation, including the decision points for login status and payment validation.

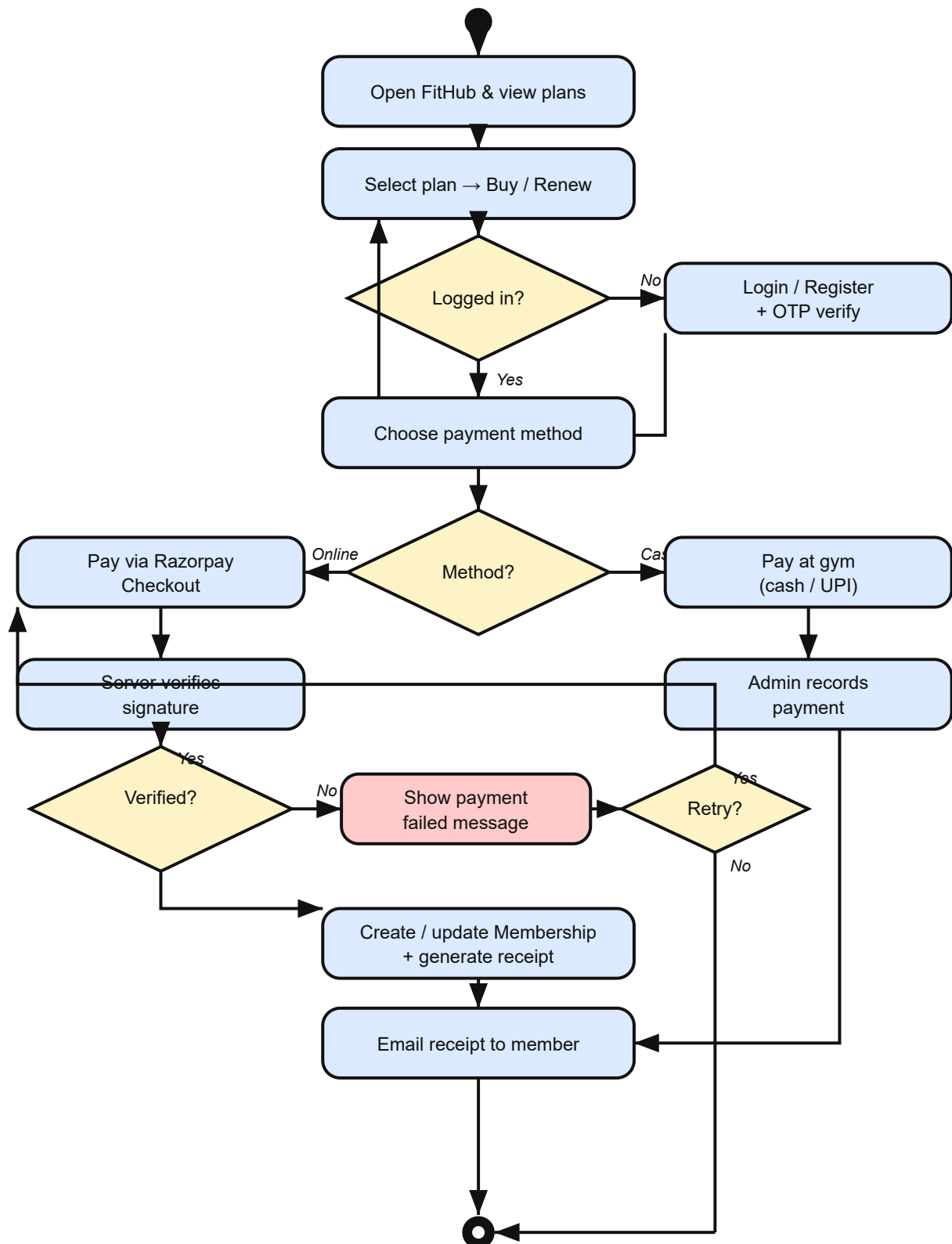


Fig 3.4.5: Activity Diagram – Buy Membership & Payment

